# 🧠 Interview TA Bot — Interview Support Agent

> **Enterprise-grade, AI-powered real-time interview assistant built as a native Windows desktop app.**
> Powered by **Azure OpenAI GPT-4.1** for adaptive questioning, live answer evaluation, and interviewer guidance.

---

## 📌 Table of Contents



---

## 1. Project Overview

The **Interview TA Bot** is a desktop application designed to assist interviewers in real time. An interviewer can load a candidate's resume and job description (JD) into the app, and the AI engine:

- **Generates the first question** calibrated to the candidate's profile and the JD.
- **Evaluates each answer** the candidate gives (rating: `strong | partial | weak`).
- **Automatically adapts difficulty** — harder questions if the candidate performs well, easier ones if they struggle.
- **Provides the interviewer** with expected answers, reference answers, follow-up suggestions, and risk flags (e.g., `overclaiming`, `shallow_knowledge`).

- **Records system audio** via WASAPI Loopback, transcribes it with Azure STT, and uses the transcript as the candidate's answer.
- **Generates a final performance summary** scored 0–100 with `Strong Hire | Hire | No Hire` recommendation.
- **Exports** the full interview report as a PDF or DOCX.

The app runs entirely **offline from the user's perspective** — it's a packaged Electron desktop app with an embedded Python (FastAPI) sidecar process.

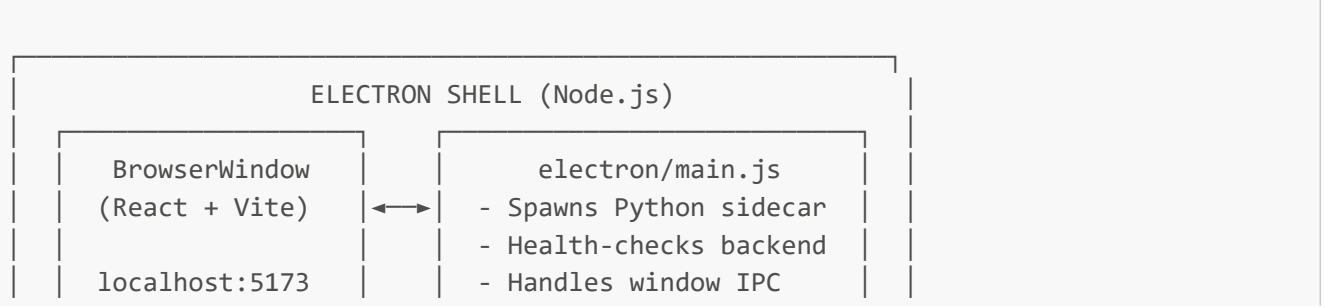
---

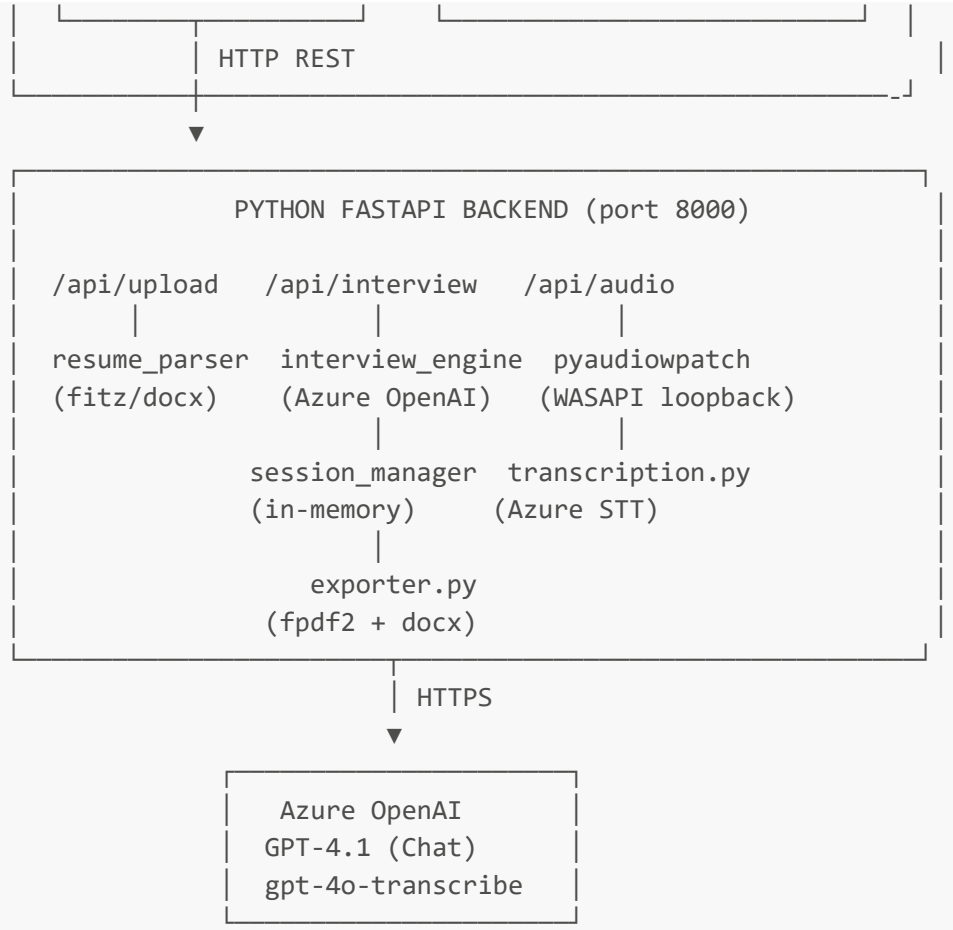
## 2. Tech Stack

| Layer | Technology | Purpose |
|---|---|---|
| Desktop Shell | **Electron 35** | Native Windows app with frameless transparent window |
| Frontend UI | **React 19 + Vite 8** | In-app UI rendered inside Electron |
| IPC Bridge | **Electron preload.js** | Secure contextBridge between renderer and main process |
| Backend API | **FastAPI + Uvicorn** | Python REST API served as a sidecar on port 8000 |
| AI Engine | **Azure OpenAI GPT-4.1** | Adaptive question generation + answer evaluation |
| Speech-to-Text | **Azure gpt-4o-transcribe** | System audio transcription |
| Audio Capture | **PyAudioWPatch (WASAPI)** | Windows loopback audio capture (captures system speaker output) |
| Data Validation | **Pydantic v2** | Strict request/response schema validation |
| Configuration | **pydantic-settings** | `.env`-based config with type safety |
| PDF Export | **fpdf2** | Programmatic PDF generation |
| DOCX Export | **python-docx** | Programmatic Word document generation |
| Resume Parsing | **PyMuPDF (fitz)** | PDF text extraction |
| Resume Parsing | **python-docx** | DOCX text extraction |
| HTTP Client | **httpx** | Async HTTP with SSL bypass for corporate proxies |
| Build / Packaging | **electron-builder** | Packages app into a Windows NSIS installer |

---

## 3. Project Architecture






---

## 4. Directory Structure

```
Interview TA Bot/
├── .env                          # Root env (legacy - desktop app has its own)
├── config.py                     # Root config (legacy)
├── cmds to exe desktop app       # Step-by-step run commands for Windows
│
└── desktop-app/                  # Main application folder
    ├── package.json              # Node.js manifest — scripts, deps, electron-
builder config
    ├── index.html                # Vite entry HTML
    ├── eslint.config.js          # ESLint configuration
    │
    ├── electron/
    │   ├── main.js               # Electron main process — app lifecycle, Python
sidecar
    │   └── preload.js            # Secure IPC bridge (contextBridge)
    │
    ├── public/
    │   ├── favicon.svg           # Brain emoji favicon
    │   └── icons.svg             # App icon set
    │
    └── python_backend/
        ├── main.py               # FastAPI app + CORS config + router registration
        ├── config.py             # Settings class (pydantic-settings) reads .env
```

```
        ├── requirements.txt      # Python dependencies
        ├── .env                  # Azure credentials (NOT committed to git)
        │
        ├── models/
        │   └── schemas.py        # All Pydantic models — enums, state, requests,
responses
        │
        ├── prompts/
        │   └── interview_prompt.py  # All LLM system prompts + message builders
        │
        ├── routers/
        │   ├── upload.py         # POST /api/upload/resume, /api/upload/jd
        │   ├── interview.py      # All interview lifecycle endpoints
        │   └── audio.py          # WASAPI loopback start/stop + transcription
        │
        └── services/
            ├── interview_engine.py  # Core AI logic — calls GPT-4.1
            ├── session_manager.py   # In-memory session store + adaptive
difficulty
            ├── resume_parser.py     # PDF/DOCX text extraction
            ├── transcription.py     # Azure STT client
            └── exporter.py          # PDF + DOCX report generation
```

---

# 5. Component Deep-Dive

## 5.1 Electron Desktop Shell

**File:** `desktop-app/electron/main.js`

Electron is the wrapper that makes the web app behave like a native desktop application.

**Key responsibilities:**

**Python sidecar management:** When the app launches, `main.js` calls `startPythonBackend()`, which uses Node's `child_process.spawn()` to start the FastAPI server (`uvicorn main:app`) as a subprocess. It reads the path to the embedded Python venv:

```
desktop-app/python_backend/venv/Scripts/python.exe
```

The Electron process captures `stdout` and `stderr` from the Python process and logs them to the console.

**Health check before UI:** Before opening the browser window, Electron polls `http://localhost:8000/docs` up to 30 times (1 second apart) to confirm the backend is ready. This prevents the UI from loading before the API is available.

**Port collision guard:** On startup it checks if port 8000 is already listening (in case the user ran the backend manually). If so, it skips spawning a second Python process.

**Frameless transparent window:** The Electron window is created with `frame: false`, `transparent: true`, and `alwaysOnTop: true`. This makes the app float over other windows — useful during an interview where the interviewer needs to see both the candidate and the tool.

**Click-through logic:** The window supports `setIgnoreMouseEvents` via IPC, letting the UI signal which areas should pass mouse clicks through to underlying windows.

**IPC Handlers:** `window-minimize`, `window-maximize`, `window-close`, `set-ignore-mouse-events` — the React frontend calls these via `window.electronAPI` (exposed by `preload.js`).

**Graceful shutdown:** On `will-quit`, Electron calls `pythonProcess.kill()` to ensure the FastAPI process is terminated.

---

## 5.2 FastAPI Python Backend

**File:** `desktop-app/python_backend/main.py`

The backend is a standard FastAPI application served by Uvicorn.

**CORS configuration:**
Allows requests from multiple origins to handle different development scenarios:

- `http://localhost:5173` — Vite dev server
- `http://localhost:8000` — Direct API access
- `file://` — Electron in production (loads HTML from local filesystem)
- `allow_origin_regex: ".*"` — Catch-all for packaged Electron builds

**Three routers are mounted:**

- `upload.router` — Prefix `/api/upload`
- `interview.router` — Prefix `/api/interview`
- `audio.router` — Prefix `/api/audio`

**Health check:** `GET /` returns `{"status": "ok"}` — this is the endpoint Electron polls.

---

## 5.3 AI Engine — `interview_engine.py`

**File:** `desktop-app/python_backend/services/interview_engine.py`

This is the brain of the application. All GPT-4.1 calls go through here.

**Client initialization (lazy singleton):**
The `AzureOpenAI` client is created only once and cached in a module-level variable `_client`. It uses `httpx.Client(verify=False)` to bypass SSL certificate inspection by corporate proxies (e.g., Zscaler).

**`_call_gpt(messages, max_retries=2)`:**
Core helper that calls GPT-4.1 with `response_format={"type": "json_object"}` — forcing the model to always return valid JSON. Has retry logic for `json.JSONDecodeError`, up to 2 retries before raising.

Settings used:

- `temperature=0.4` — Slightly creative but mostly deterministic
- `max_tokens=1500`

**`generate_first_question(resume, jd)`:**
Sends resume + JD with a special first-question prompt that tells the model "no prior Q&A exists." Returns the first question + initial evaluation stub as JSON.

**`generate_next_turn(resume, jd, last_question, candidate_answer, interview_state)`:**
The main per-turn function. Sends full context to the LLM:

- Resume + JD (for grounding)
- Last question asked
- Candidate's answer
- Interview state (question count, difficulty, recent history — compact JSON string)

Returns the full interview turn response including evaluation + next question.

**`correct_text_spelling(raw_text)`:**
Uses GPT-4.1 with `temperature=0.0` and the `SPELLING_CORRECTION_PROMPT` to fix only spelling in the candidate's transcribed answer. Crucially, it preserves stutters, filler words, and grammar exactly as spoken — making the transcript authentic.

**`generate_overall_summary(resume, jd, history)`:**
After the interview ends, sends the full Q&A history to generate a `Strong Hire | Hire | No Hire` evaluation with scores, strengths, weaknesses, and recommendation.

---

### 5.4 Session Manager

**File:** `desktop-app/python_backend/services/session_manager.py`

Manages interview state entirely **in memory** (no database). Uses a module-level dictionary:

```
_sessions: dict[str, InterviewState] = {}
```

**`create_session(resume_text, jd_text) -> session_id`:**
Generates a 12-character hex UUID (e.g., `a3f9c12d8e04`) and stores a new `InterviewState` object.

**`update_session(...)`:**
Called after each Q&A turn. It:

1. Appends a `HistoryEntry` to `state.history`
2. Increments `question_count`
3. Tracks topics covered
4. Runs the **adaptive difficulty algorithm** (see Section 7)

**`get_state_summary(session_id) -> str`:**
Returns a compact string representation of the session state for injection into the LLM prompt. Includes only the last 5 history entries (truncated to 120 chars per question) to keep token count manageable.

`save_summary(session_id, summary)`:
Stores the final `InterviewSummary` Pydantic object into the session, enabling repeated retrieval without re-generating.

---

## 5.5 Routers (API Endpoints)

**Upload Router (`routers/upload.py`)**

`POST /api/upload/resume`

- Accepts: `.pdf` or `.docx` file via multipart form
- Saves to OS temp directory (not project directory — to avoid Vite dev server hot-reload triggering)
- Calls `parse_resume()` for text extraction
- Returns: `{ filename, text, char_count }`

`POST /api/upload/jd`

- Accepts: plain text via form field
- Validates: minimum 20 characters
- Returns: `{ text, char_count }`

**Interview Router (`routers/interview.py`)**

All session lifecycle management goes through here.

| Endpoint | Method | Purpose |
|---|---|---|
| `/api/interview/start` | POST | Create session + generate first question |
| `/api/interview/next` | POST | Submit answer, receive evaluation + next question |
| `/api/interview/transcribe` | POST | Accept audio file or raw text; return corrected transcript |
| `/api/interview/state/{session_id}` | GET | Return current session state (without resume/JD text) |
| `/api/interview/history/{session_id}` | GET | Return full Q&A history |
| `/api/interview/end/{session_id}` | POST | Mark session as inactive |
| `/api/interview/summary/{session_id}` | GET | Generate/return final performance summary |
| `/api/interview/export/{session_id}/{format}` | GET | Download PDF or DOCX report |

`/transcribe` **note:** This endpoint uses `request.form()` manual parsing instead of FastAPI's automatic dependency injection. Reason: the form contains a mix of optional fields (`audio` file OR `raw_text` string), which FastAPI's strict validator can't handle gracefully. This gives better error messages and flexibility.

**Audio Router (`routers/audio.py`)**

**`POST /api/audio/start-loopback/{session_id}`:**
Uses `pyaudiowpatch` (a WASAPI-enabled fork of PyAudio for Windows) to open a loopback stream on the default speakers. Audio frames are captured in a callback and stored in `recorder_state["frames"]`. This captures whatever is playing on the system — including the interviewer's Google Meet / Zoom audio.

**`POST /api/audio/stop-loopback/{session_id}`:**
Stops the stream, assembles all frames into a WAV file, calls `transcribe_audio()`, and deletes the temp file. Returns the transcribed text.

---

## 5.6 Prompts Engineering

**File:** `desktop-app/python_backend/prompts/interview_prompt.py`

This file contains all LLM prompts — arguably the most important file in the project.

**`SYSTEM_PROMPT`:**
The main interview conductor prompt. Key design decisions:

- Forces **strict JSON output** — no markdown, no preamble, no explanation outside JSON.
- Defines adaptive logic rules inline: "If answer is STRONG → increase difficulty"
- Prevents hallucination: "NEVER hallucinate skills not present in resume, JD, or transcript"
- Provides a behavioral example (Spring Boot → REST → scaling ladder)
- Defines the exact 7-key output JSON schema with nested objects

**`SPELLING_CORRECTION_PROMPT`:**
Highly specific verbatim correction prompt. The hardest constraint is keeping stutters and repetitions exactly as spoken ("I I have" must stay "I I have"). This ensures authenticity of the transcript — critical because transcripts may be used as legal records in some enterprise settings.

**`SUMMARY_PROMPT`:**
Instructs the LLM to act as a "Senior Talent Acquisition Specialist" and produce a hiring committee-level evaluation. Returns `overall_score (0-100)` + `Strong Hire | Hire | No Hire`.

**Message builders (`build_user_message`, `build_first_question_message`, `build_summary_user_message`):**
These Python functions construct the user-role message from dynamic data. Keeping them separate from the prompts makes the code testable — you can unit-test the message format without hitting Azure.

---

## 5.7 Pydantic Schemas

**File:** `desktop-app/python_backend/models/schemas.py`

All data contracts are defined here using Pydantic v2.

**Enums:**

- `Difficulty`: `easy | medium | hard`
- `Category`: `technical | behavioral | system_design | resume_based`
- `Rating`: `strong | partial | weak`
- `RiskFlag`: `none | resume_mismatch | shallow_knowledge | overclaiming`

**Core interview state (`InterviewState`):**

```
class InterviewState(BaseModel):
    session_id: str
    resume_text: str
    jd_text: str
    question_count: int = 0
    current_difficulty: Difficulty = Difficulty.EASY
    topics_covered: list[str]
    history: list[HistoryEntry]
    consecutive_weak: int = 0
    consecutive_strong: int = 0
    is_active: bool = False
    summary: Optional[InterviewSummary] = None
```

The `consecutive_weak` and `consecutive_strong` counters directly drive the adaptive difficulty algorithm.

**API response model (`InterviewResponse`):**
Validates the complete JSON returned by GPT-4.1 on every turn — 6 nested objects ensuring type safety.

---

## 5.8 Audio — WASAPI Loopback Capture

**File:** `desktop-app/python_backend/routers/audio.py`

**What is WASAPI Loopback?**
Windows Audio Session API (WASAPI) has a loopback mode that lets you record what the system is playing through the speakers — without a physical microphone. This means the app can capture the candidate's voice from a video call (Zoom, Teams, Meet) directly from the PC's audio output.

**PyAudioWPatch:**
Standard PyAudio doesn't support WASAPI loopback. `pyaudiowpatch` is a patched fork that adds this capability. The router finds the default output device and opens a loopback stream on it.

**Callback-based capture:**
Audio is captured asynchronously via a callback function (`callback(in_data, ...)`), which appends raw audio bytes to `recorder_state["frames"]`. This is non-blocking.

**WAV assembly:**
On stop, the frames are written to a temp WAV file using Python's `wave` module with the correct sample rate and channel count read from the device info.

**Corporate proxy handling:**
The transcription HTTP call uses `verify=False` and respects `HTTPS_PROXY` env variable — common in enterprise environments.

---

## 5.9 Transcription — Azure gpt-4o-transcribe

**File:** `desktop-app/python_backend/services/transcription.py`

Sends audio bytes to Azure's `gpt-4o-transcribe-diarize` deployment using a raw `httpx.AsyncClient` POST (not the OpenAI SDK, because the STT endpoint uses a different URL format).

The endpoint URL is stored entirely in `.env` as `AZURE_STT_ENDPOINT` — it includes the deployment name and API version as query parameters (Azure STT format).

Supports any audio format (`webm`, `wav`, `mp3`) — the filename extension hints to Azure which codec to use.

---

### 5.10 Resume Parser

**File:** `desktop-app/python_backend/services/resume_parser.py`

**PDF parsing (`parse_pdf`):**
Uses `fitz` (PyMuPDF) — a fast C-extension-based PDF library. Iterates all pages, calls `page.get_text()`, and joins with newlines.

**DOCX parsing (`parse_docx`):**
Uses `python-docx`, iterates all paragraphs, filters empty ones, joins text.

**`_clean_text()`:**
Post-processing step: removes null bytes (common in PDF extraction), collapses multiple spaces to one, and reduces 3+ consecutive newlines to 2. Returns clean, LLM-ready text.

---

### 5.11 Exporter — PDF & DOCX Reports

**File:** `desktop-app/python_backend/services/exporter.py`

**`generate_pdf(summary_data, history)`:**
Uses `fpdf2` (a pure-Python PDF library). Key challenge: Unicode handling. PDF fonts in fpdf2 default to Latin-1, so the `safe_text()` helper replaces smart quotes, em-dashes, and ellipses with their ASCII equivalents before encoding. The report includes:

- Header with score + rating
- Executive Summary
- Key Strengths list
- Areas for Improvement list
- Page break → Full Q&A Transcript with per-question ratings

**`generate_docx(summary_data, history)`:**
Uses `python-docx`. Writes to a `BytesIO` buffer (no temp files) and returns bytes directly. More Unicode-safe than the PDF approach. Includes styled headings, bullet lists for strengths/weaknesses, and the full transcript with bold questions.

Both methods return `bytes` — the router streams them back as a download response with `Content-Disposition: attachment`.

---

## 6. Data Flow — End-to-End

```
1. SETUP
   User → uploads resume (PDF/DOCX) → POST /api/upload/resume
   User → pastes JD text            → POST /api/upload/jd

2. START
   Frontend → POST /api/interview/start { resume_text, jd_text }
            ← { session_id, next_question, expected_answer, reference_answer, ... }

3. INTERVIEW LOOP (repeats per question)

   Option A: System Audio Capture
     Frontend → POST /api/audio/start-loopback/{session_id}
     [candidate speaks during video call...]
     Frontend → POST /api/audio/stop-loopback/{session_id}
              ← { text: "transcribed answer" }  [via Azure STT]

   Option B: Manual Input or Mic Recording
     Frontend → POST /api/interview/transcribe (form: audio file or raw_text)
              ← { text: "spelling-corrected answer" }

   Frontend → POST /api/interview/next { session_id, candidate_answer }
            ← {
                evaluation: { rating, confidence_score, reasoning },
                next_question: { question, difficulty, category },
                expected_answer, reference_answer,
                follow_up: { should_ask, question },
                interview_guidance: { suggestion_to_interviewer, risk_flag }
              }

4. END
   Frontend → POST /api/interview/end/{session_id}
   Frontend → GET  /api/interview/summary/{session_id}
            ← { overall_score, overall_rating, strengths, weaknesses,
recommendation, ... }

5. EXPORT
   Frontend → GET /api/interview/export/{session_id}/pdf
            ← [PDF binary download]

   — OR —

   Frontend → GET /api/interview/export/{session_id}/docx
            ← [DOCX binary download]
```

---

## 7. Adaptive Difficulty Algorithm

The difficulty adapts automatically in `session_manager.update_session()` after every answer.

```
State tracked per session:
  - current_difficulty: easy | medium | hard (starts: easy)
```

```
    - consecutive_strong: int (resets on non-strong answer)
    - consecutive_weak:   int (resets on non-weak answer)

  Rules:
    After each answer:
      if rating == "strong":
          consecutive_strong += 1
          consecutive_weak = 0
      elif rating == "weak":
          consecutive_weak += 1
          consecutive_strong = 0
      else:  # partial
          both counters reset to 0

    Escalation  (consecutive_strong >= 2):
      easy   → medium
      medium → hard
      [reset consecutive_strong = 0]

    De-escalation (consecutive_weak >= 2):
      hard   → medium
      medium → easy
      [reset consecutive_weak = 0]
```

This state is also passed to the LLM via `interview_state` string, so the AI model also knows the current difficulty and can calibrate question complexity accordingly.

---

## 8. API Reference

### Upload Endpoints

| Method | Endpoint | Request | Response |
|---|---|---|---|
| POST | `/api/upload/resume` | `multipart/form-data: file (.pdf/.docx)` | `{filename, text, char_count}` |
| POST | `/api/upload/jd` | `form: jd_text (string)` | `{text, char_count}` |

### Interview Endpoints

| Method | Endpoint | Request | Response |
|---|---|---|---|
| POST | `/api/interview/start` | `{resume_text, jd_text}` | `{session_id, next_question, expected_answer, ...}` |
| POST | `/api/interview/next` | `{session_id, candidate_answer}` | `{evaluation, next_question, expected_answer,` |

| Method | Endpoint | Request | Response |
|---|---|---|---|
| | | | `follow_up, interview_guidance}` |
| POST | `/api/interview/transcribe` | `form: session_id + audio (file) OR raw_text` | `{session_id, text}` |
| GET | `/api/interview/state/{session_id}` | — | `InterviewState` (no resume/JD text) |
| GET | `/api/interview/history/{session_id}` | — | `{session_id, history[], count}` |
| POST | `/api/interview/end/{session_id}` | — | `{session_id, status: "ended"}` |
| GET | `/api/interview/summary/{session_id}` | — | `InterviewSummary` |
| GET | `/api/interview/export/{session_id}/pdf` | — | Binary PDF download |
| GET | `/api/interview/export/{session_id}/docx` | — | Binary DOCX download |

### Audio Endpoints

| Method | Endpoint | Request | Response |
|---|---|---|---|
| POST | `/api/audio/start-loopback/{session_id}` | — | `{status: "started", device}` |
| POST | `/api/audio/stop-loopback/{session_id}` | — | `{session_id, text}` |

## 9. Configuration & Environment Variables

**File:** `desktop-app/python_backend/.env`

```
# Azure OpenAI — GPT-4.1
AZURE_OPENAI_ENDPOINT=https://<your-resource>.openai.azure.com/
AZURE_OPENAI_API_KEY=<your-key>
AZURE_OPENAI_API_VERSION=2024-12-01-preview
AZURE_GPT41_DEPLOYMENT=gpt-4.1

# Azure STT — gpt-4o-transcribe
AZURE_STT_ENDPOINT=https://<your-resource>.openai.azure.com/openai/deployments/gpt-4o-transcribe-diarize/audio/transcriptions?api-version=2024-06-01
AZURE_STT_DEPLOYMENT=gpt-4o-transcribe-diarize
AZURE_STT_API_KEY=<your-key>

# App
APP_HOST=0.0.0.0
APP_PORT=8000
APP_DEBUG=true
```

Configuration is managed by `pydantic-settings BaseSettings` class with `@lru_cache()` — settings are loaded once at startup and cached as a singleton via `get_settings()`.

---

# 10. Running the Application

## Prerequisites

- Windows 10/11
- Node.js 18+
- Python 3.12+ (with the venv set up at `desktop-app/python_backend/venv/`)

## First-time Setup

**Python backend:**

```
cd desktop-app/python_backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

**Node frontend:**

```
cd desktop-app
npm install
```

**Configure `.env`:**
Create `desktop-app/python_backend/.env` with your Azure credentials (see Section 9).

## Running (Development)

```
# 1. Navigate to desktop-app
cd desktop-app

# 2. Kill any stale processes (optional but recommended after a crash)
taskkill /IM node.exe /F
taskkill /IM python.exe /F
taskkill /IM electron.exe /F

# 3. Launch — starts Vite + Electron + Python backend together
npm.cmd run desktop
```

> **Why `npm.cmd` instead of `npm`?**
> On Windows with PowerShell, `npm` resolves to `npm.ps1` which may be blocked by execution policy. `npm.cmd` bypasses this restriction.

The `desktop` script runs: `concurrently "npm run dev" "wait-on http://localhost:5173 && npm run electron:start"` — Vite starts first, and Electron waits until Vite's dev server is ready before launching.

### Building for Production (Windows Installer)

```
npm.cmd run build:electron
```

This runs `vite build` then `electron-builder --win`, producing a Windows NSIS installer in `dist/`.

---

## 11. Key Design Decisions

**Why Electron + Python instead of a pure web app?**
WASAPI loopback capture requires Windows system-level API access, which is impossible from a browser. Electron gives native OS access while keeping the UI in familiar React/HTML.

**Why FastAPI as a sidecar instead of using Node.js for everything?**
The AI/ML ecosystem (Azure OpenAI SDK, PyMuPDF, fpdf2, pyaudiowpatch) is Python-native. Running a FastAPI sidecar lets us use best-in-class Python libraries while keeping the desktop shell in Electron.

**Why in-memory session storage?**
Interviews are ephemeral, single-user sessions. A database would add setup complexity and latency with no benefit. Sessions exist only as long as the app is running — the export feature (PDF/DOCX) covers the need to persist results.

**Why `response_format: json_object` in every GPT call?**
Without this, GPT-4.1 occasionally wraps JSON in markdown code fences or adds preamble text ("Sure! Here is the evaluation:..."), which crashes JSON parsing. The json_object mode guarantees clean output.

**Why `temperature=0.4` for interview questions but `0.0` for spelling correction?**
For question generation, some creativity is desirable — we don't want every interview to be identical. For spelling correction, we want 100% deterministic output — the model should not "interpret" or creatively alter the transcript.

**Why `verify=False` in httpx?**
Corporate environments (like Stratacent) often use SSL inspection proxies (Zscaler). These replace the server's SSL certificate with the proxy's own certificate, which Python's default SSL verification rejects. `verify=False` bypasses this.

**Why does the `transcribe` endpoint parse the form manually?**
FastAPI's dependency injection is strict — declaring both an optional `UploadFile` and an optional string in the same form confuses the validation layer. Manual `request.form()` parsing gives full control over what's present and better error messages.

---

## 12. Interview Talking Points

Use these to explain the project confidently in interviews:

**"What problem does this solve?"**

> "Interviewers often struggle to evaluate candidates objectively — they forget to follow up on weak answers, don't adapt question difficulty, and have no structured record of the interview. This tool acts as a real-time AI co-pilot that handles all of that automatically."

**"What's the most technically interesting part?"**

> "The adaptive difficulty engine. It uses a state machine where difficulty escalates after 2 consecutive strong ratings and de-escalates after 2 weak ones. Both the Python session manager and the GPT-4.1 prompt are aware of this state — the LLM receives the current difficulty in its context so it calibrates question complexity accordingly."

**"How do you handle the AI producing inconsistent output?"**

> "I force `response_format: json_object` on every GPT call, which constrains the model to return only valid JSON. I also have retry logic — if the JSON is malformed on the first attempt, it retries up to 2 times before raising an exception."

**"Why use Electron? Why not just a web app?"**

> "WASAPI loopback — capturing system audio on Windows requires native OS API access that browsers don't allow. Electron gives me a native shell where I can run a Python subprocess that uses PyAudioWPatch for loopback capture, while keeping the UI in React."

**"How does the speech-to-text work?"**

> "Two paths: if the interviewer uses the loopback feature, pyaudiowpatch captures system audio (what the candidate is saying over Zoom/Teams), assembles it into a WAV, and sends it to Azure's gpt-4o-transcribe model. Separately, a spelling correction pass using GPT-4.1 at temperature=0 fixes only word-level spelling, preserving the natural speech patterns like stutters and filler words."

**"What would you improve if you had more time?"**

> "Replace in-memory session storage with SQLite for persistence across restarts. Add a frontend React component to display the live evaluation feedback visually. Add support for multiple concurrent interview sessions. Potentially containerize the Python backend with Docker for cross-platform deployment — right now WASAPI loopback is Windows-only."

---

*README generated from source code analysis. Author: Chinmay Shete. Project: Interview Support Agent v1.0.0*